

boilert

Water Boiler Heat Monitor

Installation Manual

Hardware – Software – Commissioning

Version 1.3.1

Date July 2026

Source <https://github.com/guycorbaz/boilert>

For a Raspberry Pi with the official 7 " touchscreen
and DS18B20 1-Wire temperature sensors

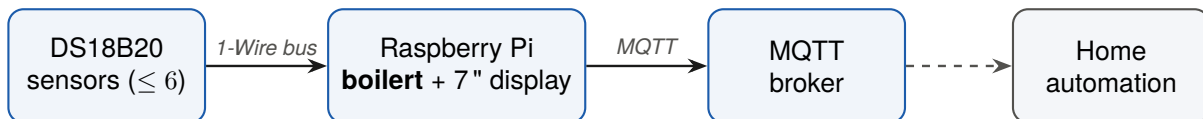
Contents

1	Introduction	3
1.1	System architecture	3
1.2	How the energy is calculated	3
2	Bill of materials	3
3	Hardware installation	4
3.1	Sensor placement on the tank	4
3.2	Wiring the 1-Wire bus	5
4	Raspberry Pi preparation	6
4.1	Operating system	6
4.2	Enable the 1-Wire interface	6
4.3	Verify that the sensors are detected	6
4.4	Identify each sensor's position	6
5	Software installation	7
5.1	Install the Rust toolchain	7
5.2	Build boilert	7
6	Configuration	7
6.1	Example	8
6.2	Reference	8
7	MQTT integration	9
8	Autostart with systemd	9
9	Operation	10
9.1	The two screens	10
9.2	Status indicators	10
9.3	History persistence	10
10	Troubleshooting	11
10.1	Replacing a sensor	11

1 Introduction

boilert estimates the amount of thermal energy stored in a cylindrical domestic hot water tank. Between one and six DS18B20 temperature sensors are attached to the tank wall, evenly spaced along its height; each sensor is assumed to represent an equal horizontal slice of the water volume. The application displays the temperatures, the computed energy and their 24-hour history on the touchscreen, and publishes every measurement to an MQTT broker for integration with a home automation system.

1.1 System architecture



1.2 How the energy is calculated

Each valid sensor i represents a slice of volume V/N (tank volume divided by the number of sensors). The stored energy above the cold-water reference temperature T_{ref} is:

$$E \text{ [kWh]} = \frac{1}{1000} \sum_{i=1}^N \frac{V}{N} \cdot \max(0, T_i - T_{\text{ref}}) \cdot c$$

where $c = 1.162 \text{ Wh/(l} \cdot \text{K)}$ for water. Accuracy therefore depends directly on *even sensor spacing* and *good thermal contact* — the two points this manual insists on.

2 Bill of materials

Qty	Item	Remark
1	Raspberry Pi (3B+ or newer recommended)	With power supply
1	Official 7" touchscreen (800 × 480)	Or any 800×480 display
1–6	DS18B20 sensors, waterproof probe version	Stainless sleeve, 1–3 m cable
1	Resistor 4.7 kΩ, 1/4 W	Single pull-up for the whole bus
–	Wire, 3 conductors (e.g. 0.5 mm ²)	Length: tank → Pi
–	Screw terminals or solder + heat-shrink	Bus junctions
1	Aluminium adhesive tape + thermal paste	Sensor mounting
–	Cable ties / stainless hose clamps	Sensor fixation
1	MicroSD card (≥ 8 GB)	Raspberry Pi OS

Note

An MQTT broker (e.g. Mosquitto) must be reachable on the network. It can run on the same Raspberry Pi, on a NAS, or inside Home Assistant.

Dedicated interface PCB

Instead of the hand-wired pull-up resistor and screw terminals, you can use the **1-Wire interface board** designed for this project: https://github.com/guycorbaz/1wire_raspi_pcb. It is an open-hardware KiCad 9 project that plugs onto the 40-pin GPIO header and provides Molex Micro-Fit 3.0 connectors for the probes and an external supply, with the bus pull-up and supply filtering already on board.

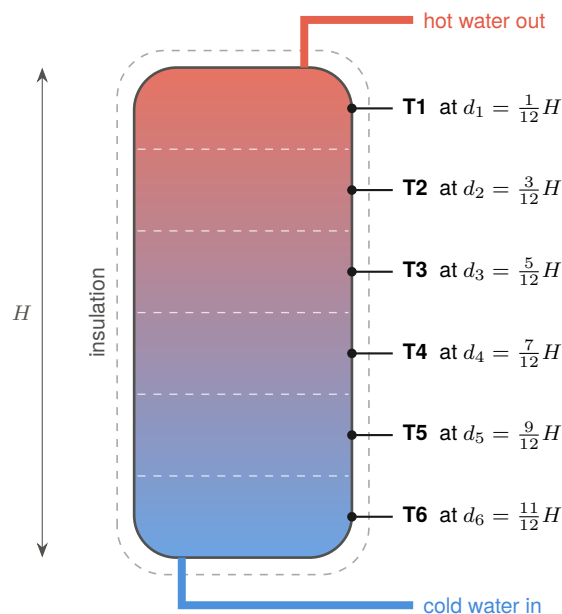
3 Hardware installation

3.1 Sensor placement on the tank

The energy model divides the tank into N horizontal slices of equal volume. Each sensor must measure the temperature at the *center* of its slice: for a tank of height H and N sensors, sensor i (counted from the top) is placed at depth

$$d_i = \frac{(2i - 1) H}{2N} \quad \text{from the top of the water volume.}$$

For the typical case $N = 6$: $d = H/12, 3H/12, 5H/12, 7H/12, 9H/12, 11H/12$.



Mounting each sensor:

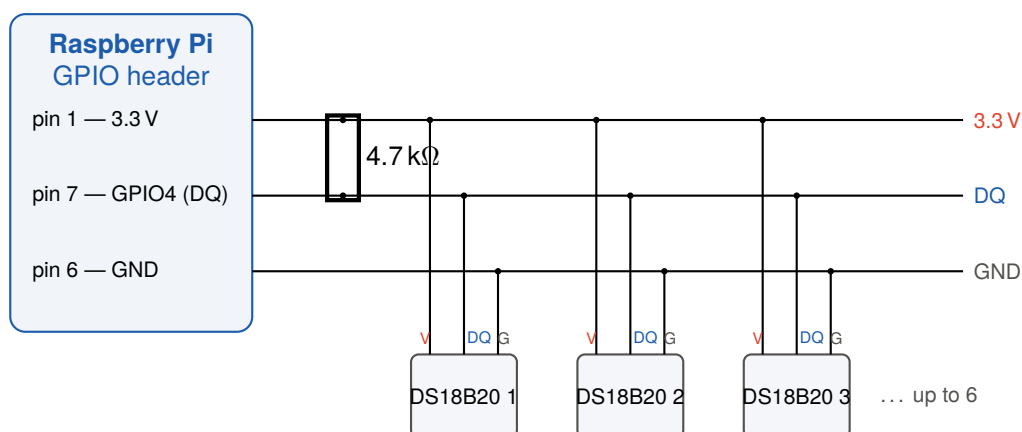
1. Remove the insulation locally to expose the bare steel wall.
2. Apply a dab of thermal paste, press the stainless probe flat against the wall (along the wall, not perpendicular).
3. Fix it with aluminium adhesive tape (which also spreads the heat), a hose clamp or a cable tie around the tank.
4. Put the insulation back over the sensor. A probe left in open air reads the room, not the water.

Warning

The order matters: the *first* sensor in `config.toml` must be the *highest* probe on the tank, the last one the lowest. The stratification display and the energy model both rely on this top-to-bottom ordering.

3.2 Wiring the 1-Wire bus

All sensors share the same three wires: **GND**, **DQ** (data) and **3.3 V**. A single 4.7 k Ω pull-up resistor between DQ and 3.3 V serves the whole bus. Waterproof DS18B20 probes almost always use: red = 3.3 V, black = GND, yellow (or white/blue) = DQ.



Wiring recommendations:

- **Powered mode, not parasite mode:** always connect the red wire to 3.3 V. Parasite power (VDD tied to GND) is less reliable with several sensors.
- **One single pull-up** of 4.7 k Ω , at the Pi end of the bus. Do not add one per sensor.
- **Daisy chain** the sensors along one cable run rather than a star topology; a linear bus is much more robust. Up to ~10 m of total cable is unproblematic for 6 sensors.
- Use 3.3 V, *never* 5 V: GPIO4 is not 5 V tolerant.

Using the interface PCB

With the companion board (https://github.com/guycorbaz/1wire_raspi_pcb) the wiring above is already done for you: plug the board onto the GPIO header and connect each probe to a Micro-Fit connector. The pull-up resistor of this section must *not* be added again — it is on the board.

4 Raspberry Pi preparation

4.1 Operating system

Install Raspberry Pi OS (with desktop — the application opens a window on the display server) using Raspberry Pi Imager, and boot the Pi.

4.2 Enable the 1-Wire interface

Add the overlay to the boot configuration (`/boot/firmware/config.txt` on current Raspberry Pi OS, `/boot/config.txt` on older releases):

```
sudo sh -c 'echo "dtoverlay=w1-gpio" >> /boot/firmware/config.txt'
sudo reboot
```

4.3 Verify that the sensors are detected

After reboot, each DS18B20 appears as a directory beginning with 28-:

```
ls /sys/bus/w1/devices/
# 28-3ce1d443a1b2 28-3ce1d443c4f1 ... w1_bus_master1

cat /sys/bus/w1/devices/28-3ce1d443a1b2/w1_slave
# 72 01 4b 46 7f ff 0e 10 57 : crc=57 YES
# 72 01 4b 46 7f ff 0e 10 57 t=23125      <- 23.125 degrees C
```

If fewer sensors appear than are connected, check the bus junctions and the pull-up resistor first — a missing pull-up is the most common cause.

4.4 Identify each sensor's position

The 1-Wire IDs are factory-set and carry no position information, so you must map each ID to its physical location once:

1. Open a terminal and watch one candidate ID:

```
watch -n 1 cat /sys/bus/w1/devices/28-*/w1_slave
```

2. Warm *one* probe with your fingers (or a cup of warm water) before mounting, and note which ID rises.
3. Repeat for each probe, filling a table like this:

Position	Config name	1-Wire ID	Depth
Top (1)	T1	28-_____	<i>H</i> /12
(2)	T2	28-_____	3 <i>H</i> /12
(3)	T3	28-_____	5 <i>H</i> /12
(4)	T4	28-_____	7 <i>H</i> /12
(5)	T5	28-_____	9 <i>H</i> /12
Bottom (6)	T6	28-_____	11 <i>H</i> /12

Tip

Do the identification *before* mounting the probes on the tank and label each cable (T1...T6) with tape. It is much harder afterwards.

5 Software installation

5.1 Install the Rust toolchain

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

5.2 Build boilert

```
git clone https://github.com/guycorbaz/boilert.git
cd boilert
cargo build --release --features pi
```

The `pi` feature enables real sensor reading; without it the application generates simulated data (useful for testing the display).

Note

The first build compiles the Slint UI toolkit and can take 30–60 minutes on a Raspberry Pi. On models with 1 GB of RAM, limit parallelism with `cargo build -release -features pi -j 2`.

The binary is produced at `target/release/boilert`.

6 Configuration

The application reads a TOML file given as its first argument (default: `config.toml` in the working directory):

```
./target/release/boilert /home/pi/boilert/config.toml
```

6.1 Example

```
# Persist the 24 h histories across restarts (JSON)
history_file = "boilert-history.json"

[mqtt]
host = "mqtt.home.arpa"
port = 1883
base_topic = "boilert/sensors"
publish_interval_s = 30

[boiler]
volume_l = 500.0           # Total volume in liters
reference_temp_c = 15.0    # Cold water reference
energy_coefficient = 1.162 # Wh/(l K), water

[ui]
fullscreen = true          # kiosk mode on the touchscreen

# From the TOP of the tank to the BOTTOM (1 to 6 sensors)
[[sensors]]
name = "T1"
id = "28-3ce1d443a1b2"

[[sensors]]
name = "T2"
id = "28-3ce1d443c4f1"

# ... T3 to T6
```

6.2 Reference

Key	Default	Description
history_file	boilert-history.json	Where the 24 h histories are persisted.
mqtt.host	—	Broker hostname or IP.
mqtt.port	—	Broker port (usually 1883).
mqtt.base_topic	—	Prefix of all published topics.
mqtt.publish_interval_s	30	Seconds between publications.
boiler.volume_l	—	Tank volume in liters (> 0).
boiler.reference_temp_c	—	Cold-water baseline for the energy.
boiler.energy_coefficient	—	1.162 Wh/(l·K) for water (> 0).
ui.fullscreen	false	Fullscreen kiosk mode.
sensors[].name	—	Display name and MQTT sub-topic.
sensors[].id	—	1-Wire device ID (28-...).

The configuration is validated at startup: 1 to 6 sensors, positive volume, coefficient and interval. Errors are printed and the application refuses to start, so a typo cannot go unnoticed.

7 MQTT integration

Every `publish_interval_s` seconds the application publishes **retained** messages with QoS 1 (client id `boilert-<pid>`):

Topic	Content	Payload
<code>{base_topic}/{name}</code>	Temperature of one sensor	°C, 2 decimals
<code>{base_topic}/energy</code>	Stored energy	kWh, 3 decimals
<code>{base_topic}/status</code>	Availability (last will)	online / offline

A sensor that fails to read is *not* published (its previous retained value remains on the broker); for the energy figure its slice is interpolated from the neighboring valid sensors. `offline` is set automatically by the broker if the connection drops (MQTT last will).

Verify from any machine:

```
mosquitto_sub -h mqtt.home.arpa -t 'boilert/sensors/#' -v
# boilert/sensors/status online
# boilert/sensors/T1 57.81
# boilert/sensors/energy 16.402
# ...
```

8 Autostart with systemd

A sample unit is provided in `deploy/boilert.service`. Adjust `User`, `WorkingDirectory` and `ExecStart` to your paths, then:

```
sudo cp deploy/boilert.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now boilert
journalctl -u boilert -f          # follow the logs
```

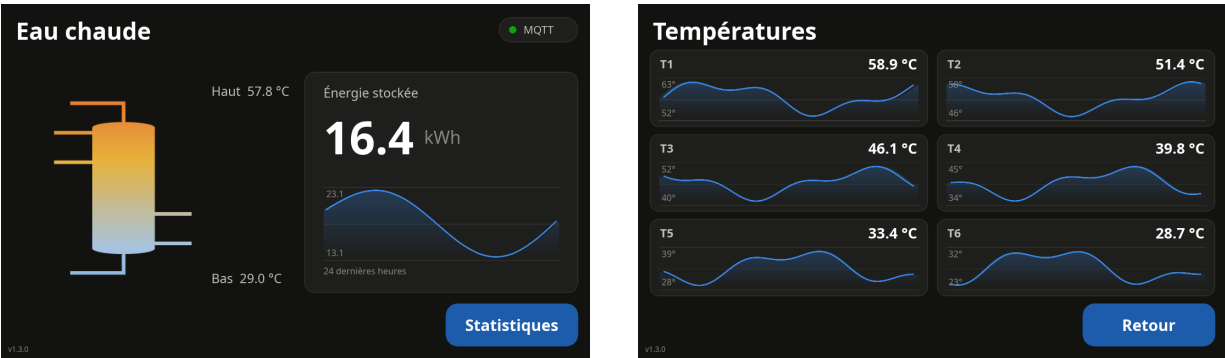
The service restarts automatically on failure. Log verbosity is controlled by the `RUST_LOG` environment variable in the unit (`info` by default; use `debug` while commissioning).

Tip

For a clean kiosk: set `fullscreen = true` in the `[ui]` section, hide the mouse cursor (unclutter), and disable screen blanking in the Raspberry Pi OS display settings.

9 Operation

9.1 The two screens



Dashboard (left): tank drawing colored by the measured stratification, current top/bottom temperatures, stored energy with its 24-hour history graph. **Statistics** (right): one card per sensor with the current value and its auto-scaled 24-hour graph. The buttons at the bottom right switch between the screens.

9.2 Status indicators

Indicator	Meaning
MQTT chip, green dot	Connected to the broker.
MQTT chip, red dot	Broker unreachable; measurements continue, publications resume automatically on reconnection.
Amber “défaut sonde” chip	At least one sensor failed its last reading.
Sensor card with red border/value	This sensor is failing; the shown value is the last valid one.
Gap in a history graph	Sensor failure or application downtime during that period.

9.3 History persistence

The 24-hour histories (one point per 15 minutes, per sensor plus the stored energy) are saved in `history_file` and restored at startup, shifted by the downtime. Deleting the file simply restarts the graphs from scratch. If the *number* of sensors in the configuration changes, the history is also restarted from scratch — this is intentional.

10 Troubleshooting

Symptom	Check
No 28-* directory at all	1-Wire overlay present in <code>config.txt</code> ? Rebooted? Pull-up resistor fitted? DQ on GPIO4 (pin 7)?
Some sensors missing	Bus junctions (a single bad crimp drops everything after it); total cable length; star topology (convert to daisy chain).
Sensor shown in red, <code>crc=...</code> NO in <code>w1_slave</code>	Electrical noise or loose contact: check the junction of that probe, keep the bus away from mains cables.
Temperatures look swapped	Sensor order in <code>config.toml</code> does not match the physical top-to-bottom order; redo the mapping of section 4.4.
MQTT chip stays red	<code>mqtt.host/port</code> correct? Broker reachable (ping, <code>mosquitto_sub</code>)? Firewall?
Energy value seems wrong	<code>volume_l</code> and <code>reference_temp_c</code> correct? All sensors reading plausibly (Statistics screen)? Thermal contact (a probe reading the room instead of the wall pulls the value down)?
Window has no titlebar / cannot be closed on a desktop PC	Intentional (kiosk design). Close with <code>Alt+F4</code> or <code>kill</code> .

Useful commands while commissioning:

```
journalctl -u boilert -f          # application logs
cat /sys/bus/w1/devices/28-*/w1_slave # raw sensor readout
mosquitto_sub -h <broker> -t 'boilert/#' -v # what the broker sees
RUST_LOG=debug ./boilert config.toml # verbose foreground run
```

10.1 Replacing a sensor

A new DS18B20 has a new 1-Wire ID:

1. Identify the new probe's ID (section 4.4).
2. Replace the `id` of the corresponding `[[sensors]]` entry in `config.toml` — keep the same name and position.

3. Restart the service: `sudo systemctl restart boilert`.

Since the sensor *count* is unchanged, the history is preserved; only the replaced sensor's curve continues with the new probe's data.